

Supplementary Materials for “changepointGA: An R package for Fast Changepoint Detection via Genetic Algorithms”

by Mo Li and QiQi Lu

S1. At most one changepoint (AMOC) detection using changepointGA

Assuming a single changepoint ($m = 1$) at time $\tau \in \{2, \dots, N - 1\}$ in the first order autoregressive (AR(1)) series $\{X_t\}_{t=1}^N$ and the changepoint τ partitions the process into two regimes. Such a problem often involves distinguishing between two situations: the time series contains no changepoint, or there is exactly one changepoint at an unknown time τ . Hypothesis testing is one of the prevalent methods for addressing AMOC detection problems. Specifically, the likelihood ratio test is commonly employed to discern the presence of a changepoint and ascertain its location if one indeed exists. In general, the test statistic takes the form

$$\Lambda_{\max} = \max_{\ell \leq \tau \leq h} \Lambda(\tau) \quad \text{and} \quad \Lambda(\tau) = -2 \left(\log(L_0(\hat{\mu}_0)) - \log(L_\tau(\hat{\mu}_1, \hat{\mu}_2)) \right), \quad (1)$$

where $\hat{\mu}_0 = \hat{\beta}_0$ is the maximum likelihood estimator (MLE) for time series mean under the null hypothesis and $L_0(\hat{\mu}_0)$ is the corresponding model likelihood. $L_\tau(\hat{\mu}_1, \hat{\mu}_2)$ is the likelihood function under the alternative hypothesis, where $\hat{\mu}_1 = \hat{\beta}_0$ and $\hat{\mu}_2 = \hat{\beta}_0 + \hat{\beta}_1$ are the MLEs for the means of the two segments separated by changepoint $t = \tau$. To address the issue of volatility in the test statistic near the boundaries, a cropped $\Lambda_{\max}$ is commonly used. Instead of maximizing over the entire range $1 < \tau \leq N$, the admissible changepoint locations are truncated between ℓ (close to 1) and h (close to N). [Robbins et al. \(2011\)](#) show that

$$\Lambda_{\max} \xrightarrow{D} \sup_{\ell \leq t \leq h} \frac{B^2(t)}{t(1-t)}, \quad (2)$$

where $\{B(t)\}_{t=0}^1$ denotes a standard Brownian bridge process and $\xrightarrow{D}$ signifies convergence in distribution, and provide the p-value calculation for this test.

To compute the test statistic $\Lambda_{\max}$, one maximizes $\Lambda(\tau)$ over all admissible changepoints $\ell \leq \tau \leq h$. This procedure involves fitting alternative models a total of $h - \ell + 1$ times. In cases with shorter time series, the number of candidate changepoints is limited, making the hypothesis testing procedure efficient. However, for longer time series with large sample sizes, the candidate set of changepoints expands, necessitating numerous model fittings and significantly increasing computational costs. In a recent study by [Li et al. \(2026\)](#), under the marginalized transition model framework, they proposed a genetic algorithm approach for the AMOC problem. Since the $\log(L_0(\hat{\mu}_0))$ part in Equation (1) remains constant for every admissible τ , the objective function only fits the model under the null hypothesis once and inputs the value as an argument $\log L_0$. This approach intelligently searches for the optimal solution that maximizes Λ_τ without exhaustively fitting alternative models for every candidate changepoint. The optimized $\Lambda_{\max}$ value is then compared with the critical value from the limiting distribution in Equation (2) to facilitate the decision-making in hypothesis testing.

We designed the chromosome representation and the objective function to maximize $\Lambda(\tau)$ under the alternative hypothesis. The related R function used here (recalling that our default optimization is to minimize the objective function), is defined as follows:

```
lambda_tau <- function(chromosome, plen = 0, Xt, XMat, logL0) {
```

```

N <- length(Xt)
m <- chromosome[1]
tau <- chromosome[2]
if (tau < floor(0.05 * N) | tau > ceiling(0.95 * N)) {
  Lambdatau <- -2 * (logL0 + 1e6)
} else {
  cpt <- c(rep(0, tau), rep(1, N - tau))
  DesignX <- cbind(XMat, cpt)
  fit <- arima(Xt, order = c(1, 0, 0), xreg = DesignX, include.mean = FALSE)
  logLa <- fit$loglik
  Lambdatau <- -2 * (logL0 - logLa)
}
return(-Lambdatau)
}

```

The `changepointGA` package offers users the flexibility to define custom genetic operators. The functions `amoc_population`, `amoc_selection`, `amoc_crossover`, and `amoc_mutation` from our `changepointGA` can be treated as the custom genetic operators and be integrated into arguments in `cptga` and `cptgaisl`. Detailed explanations and illustrations of these custom genetic operator functions can be found in the function help documentation or the supplementary materials of [Li et al. \(2026\)](#). With the estimated $\log L_0(\hat{\mu}_0)$ under the null hypothesis, we can run the genetic algorithm to approximately but efficiently find the $\hat{\tau}$ to maximize $\Lambda(\tau)$.

```

N <- 1000
XMatT <- matrix(1, nrow = N, ncol = 1)
Xt <- ts_sim(
  Ts = N, beta = 0.5, XMat = XMatT, sigma = 1, phi = 0.5, theta = NULL, Delta = 2,
  CpLoc = 500, seed = 1234
)
DesignXEst <- XMatT
fit0 <- arima(Xt, order = c(1, 0, 0), xreg = DesignXEst, include.mean = FALSE)
logL0 <- fit0$loglik
res.changepointGA <- cptga(
  ObjFunc = lambda_tau, N = N, popSize = 200,
  pcrossover = 0.95, pmutation = 0.3, pchangept = 10 / N,
  popInitialize = "amoc_population",
  selection = "amoc_selection",
  crossover = "amoc_crossover",
  mutation = "amoc_mutation",
  Xt = Xt, XMat = XMatT, logL0 = logL0
)
summary(res.changepointGA)

#> #####
#> #           Changepoint Detection via GA           #
#> #####
#> Settings:
#> Population size           = 200
#> Number of generations    = 5057
#> Crossover probability     = 0.95
#> Mutation probability     = 0.3
#> Changepoint probability  = 0.01
#> minDist                  = 1
#> Task mode                 = cp
#> Parallel Usage           = FALSE

```

```

#> Seed = NULL
#>
#> ##### GA results #####
#> Optimal Fitness value = -124.5826
#> Optimal Solution:
#> Number of Changepoints = 1
#> Changepoints Locations = 486

res.Island.changepointGA <- cptgaisl(
  ObjFunc = lambda_tau, N = N, popSize = 200, numIslands = 5,
  pcrossover = 0.95, pmutation = 0.3, pchangept = 10 / N,
  popInitialize = "amoc_population",
  selection = "amoc_selection",
  crossover = "amoc_crossover",
  mutation = "amoc_mutation",
  Xt = Xt, XMat = XMatT, logL0 = logL0
)
summary(res.Island.changepointGA)

#> #####
#> # Changepoint Detection via Island Model GA #
#> #####
#> Settings:
#> Population size = 200
#> Number of Island = 5
#> Island size = 40
#> Number of generations = 5000
#> Number of migrations = 100
#> Crossover probability = 0.95
#> Mutation probability = 0.3
#> Changept probability = 0.01
#> minDist = 1
#> Task mode = cp
#> Parallel Usage = FALSE
#> Seed = NULL
#>
#> ##### Island Model GA results #####
#> Optimal Fitness value = -124.5826
#> Optimal Solution:
#> Number of Changepoints = 1
#> Changepoints Locations = 486

fit.Island.changepointGA <- res.Island.changepointGA@overbestfit
tau.Island.changepointGA <- res.Island.changepointGA@overbestchrom
pvaluecalc <- function(x, ell, h) {
  res <- sqrt(x * exp(-x) / 2 * pi) * (1 - 1 / x) * log(((1 - ell) * h) / (ell * (1 - h))) + 4 / x
  return(res)
}
AMOCpvalue <- pvaluecalc(x = -fit.Island.changepointGA, ell = 0.05, h = 0.95)
AMOCpvalue

#> [1] 0.0321072

```

From the analysis above, both the `cptga` and `cptgaisl` functions identified a single changepoint at $\hat{\tau} = 486$ with $\Lambda_{\max} = 124.583$. This corresponds to a p-value of 0.032 (less than the significance level of $\alpha = 0.05$) via the calculation method for likelihood ratio test in [Robbins et al. \(2011\)](#). Therefore, we conclude that the identified changepoint at $\hat{\tau} = 486$ introduces a significant mean structural change.

S2. Objective functions used in the simulation study and applications

```

library(inline)
library(RcppArmadillo)

ARIMA_BIC_GA <- cxxfunction(
  signature(tauCltc_ = "numeric", XMat_ = "matrix", Xt_ = "numeric"),
  body = '
    arma::vec tauCltc = as<arma::vec>(tauCltc_);
    arma::mat XMat = as<arma::mat>(XMat_);
    arma::vec Xt = as<arma::vec>(Xt_);
    int N = Xt.n_elem;

    tauCltc[0] = 0.0;
    std::vector<int> tau;
    for (int i = 0; i < tauCltc.n_elem; ++i) {
      if (tauCltc[i] == 1.0) tau.push_back(i + 1); // 1-based indexing
    }

    arma::mat DesignX = XMat;
    int m = tau.size();

    if (m > 0) {
      std::vector<int> tmptau = tau;
      tmptau.push_back(N + 1);
      std::sort(tmptau.begin(), tmptau.end());
      tmptau.erase(std::unique(tmptau.begin(), tmptau.end()), tmptau.end());

      arma::mat CpMat = arma::zeros<arma::mat>(N, m);
      for (int i = 0; i < m; ++i) {
        int start = tmptau[i];
        int end = std::min(tmptau[i + 1] - 1, N);
        for (int j = start - 1; j <= end - 1; ++j) {
          CpMat(j, i) = 1.0;
        }
      }
      DesignX = arma::join_rows(XMat, CpMat);
    }

    arma::vec betahat;
    bool success = arma::solve(betahat, DesignX, Xt);
    if (!success) return wrap(1e6);

    arma::vec u = Xt - DesignX * betahat;
    arma::vec u_lag = u.subvec(0, N - 2);
    arma::vec u_now = u.subvec(1, N - 1);

    double denom = arma::dot(u_lag, u_lag);
    if (denom == 0.0) return wrap(1e6);

    double phi_hat = arma::dot(u_now, u_lag) / denom;
    arma::vec eps_hat = u_now - phi_hat * u_lag;
    double sigma2_hat = arma::mean(arma::square(eps_hat));

    double BIC = (N - 1) * std::log(sigma2_hat) + (2 * m + 3) * std::log(N - 1);
    return wrap(-BIC);
  '

```

```

    ',
    plugin = "RcppArmadillo"
  )

lm_search_bic <- function(chromosome, plen = 0, XMat, Xt) {
  N <- length(Xt)
  m <- chromosome[1]
  tau <- chromosome[2:length(chromosome)]

  if (m == 0) {
    DesignX <- XMat
  } else {
    tau <- tau[tau >= 1 & tau < N]
    tmptau <- unique(c(tau, N))
    CpMat <- matrix(0, nrow = N, ncol = length(tmptau) - 1)
    for (i in 1:NCOL(CpMat)) {
      CpMat[(tmptau[i] + 1):tmptau[i + 1], i] <- 1
    }
    DesignX <- cbind(XMat, CpMat)
  }

  fit <- lm(Xt ~ DesignX - 1)
  BIC.obj <- BIC(fit) + m*log(N)

  return(BIC.obj)
}

ar_bic_mean <- function(chromosome, plen = 1, XMat, Xt) {
  N <- length(Xt)

  m <- chromosome[1]
  p.order <- chromosome[2:(plen + 1)]
  tau <- chromosome[(plen + 2):length(chromosome)]

  if (m == 0) {
    DesignX <- XMat
  } else {
    tau <- tau[tau > 1 & tau < N + 1]
    tmptau <- unique(c(tau, N))
    CpMat <- matrix(0, nrow = N, ncol = length(tmptau) - 1)
    for (i in 1:NCOL(CpMat)) {
      CpMat[(tmptau[i] + 1):tmptau[i + 1], i] <- 1
    }
    DesignX <- cbind(XMat, CpMat)
  }

  fit <- arima(Xt, order = c(p.order[1], 0, 0), xreg = DesignX, include.mean = F, method = "ML")
  BIC.obj <- BIC(fit) + m*log(N)

  attr(BIC.obj, "DesignX") <- DesignX
  attr(BIC.obj, "fit") <- fit

  return(BIC.obj)
}

```

```

ar_bic_mean_trend_shift <- function(chromosome, plen = 1, XMat, Xt) {
  N <- length(Xt)

  m <- chromosome[1]
  p.order <- chromosome[2:(plen + 1)]
  tau <- chromosome[(plen + 2):length(chromosome)]

  strend <- (1:N)/N

  if (m == 0) {
    DesignX <- XMat
  } else {
    tau <- tau[tau > 1 & tau < N + 1]
    starts <- tau + 1
    ends <- c(tau[-1], N)

    cptmat <- matrix(0, nrow = N, ncol = m)
    strendmat <- matrix(0, nrow = N, ncol = m)
    for (ii in seq_len(m)) {
      idx <- starts[ii]:ends[ii]
      cptmat[idx, ii] <- 1
      strendmat[idx, ii] <- strend[idx]
    }
    DesignX <- cbind(XMat, cptmat, strendmat)
  }

  fit <- arima(Xt, order = c(p.order[1], 0, 0), xreg = DesignX, include.mean = F, method = "ML")
  BIC.obj <- BIC(fit) + m*log(N)

  attr(BIC.obj, "DesignX") <- DesignX
  attr(BIC.obj, "fit") <- fit

  return(BIC.obj)
}

```

References

- M. Li, Q. Lu, and X. Wang. Changepoint Detection in Categorical Time Series with Application to Daily Total Cloud Cover in Canada. 2026. doi: 10.48550/arXiv.2605.20621. [p1, 2]
- M. Robbins, C. Gallagher, R. Lund, and A. Aue. Mean shift testing in correlated data. *Journal of Time Series Analysis*, 32(5):498–511, 2011. doi: 10.1111/j.1467-9892.2010.00707.x. [p1, 3]

Mo Li
 University of Louisiana at Lafayette
 Department of Mathematics
 217 Maxim Doucet Hall, Lafayette, LA 70504,
 United States of America
mo.li@louisiana.edu

QiQi Lu
 Virginia Commonwealth University
 Department of Statistical Sciences and Operations Research

PO Box 843083, 1015 Floyd Ave., Richmond, VA 23284,
United States of America
qlu2@vcu.edu